

Week 2: Functions, Tests, and NumPy Review

Path Geometry and Verification

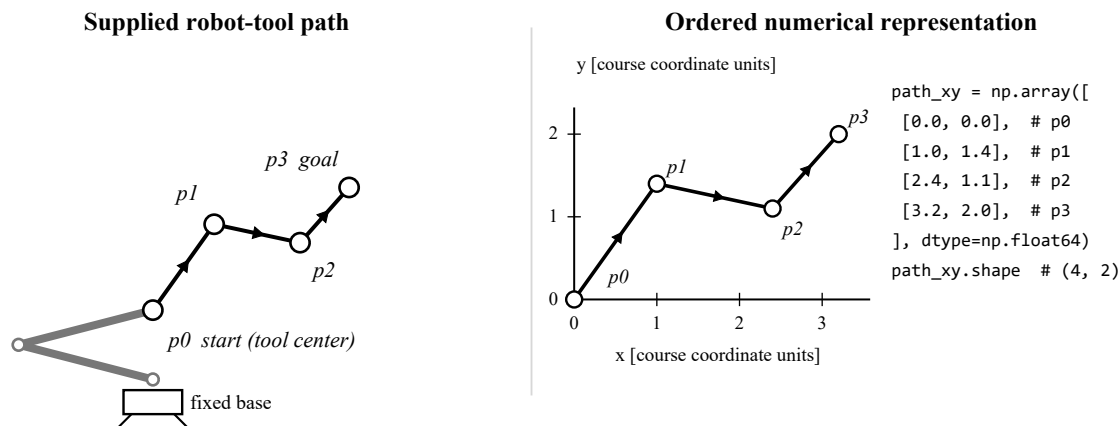


Figure 2.1. The same supplied robot-tool path has a physical interpretation and an ordered numerical representation.

The left side of Figure 2.1 gives the four waypoints a physical interpretation: they are successive tool-center positions along one supplied robot-tool path. The right side records the same ordered positions as an x-y coordinate plot and a NumPy-ready array. Row *i* stores waypoint *p_i* as (x,y); changing row order changes the path. A display can still appear plausible when code skips an endpoint, exchanges coordinates, or calculates the wrong length, so the programming question is how to publish this familiar calculation as a reusable function and support it with numerical evidence.

The chapter develops four functions in dependency order. The first calculates one segment length, the second samples one segment, the third calculates one polyline length, and the fourth applies the same meaning to a batch of paths. Python containers make the data flow visible first; NumPy then makes shapes and axes explicit without changing that meaning.

Function	Result that must be verified
<code>segment_length(...)</code>	one nonnegative distance
<code>interpolate_segment(...)</code>	exactly <i>M</i> points, including both endpoints
<code>path_length(...)</code>	sum of consecutive segment lengths
<code>path_lengths(...)</code>	one length for every path in a batch

The finished package must return useful numbers, reject invalid inputs, and leave its inputs unchanged. The final Assignment turns these claims into four implementations, 48 published checks, at least six independent student tests, a numerical summary, an animated path preview, an installable wheel, and a Git revision submitted through LMS. The preview shows supplied geometry; it does not simulate robot dynamics or establish safety.

2.1 Python review: follow the values through the program

A program first creates values and gives them names. Assignment binds a name to the result of an expression; it does not write an equation that remains true forever. A function call is also an expression: its arguments supply the parameters, the body executes, and return supplies the value used by the caller.

```
conda activate applied-programming-2026
python
```

Run these two commands in Anaconda Prompt. The first selects the existing course environment; the second opens Python. Enter the following Python examples at `>>>`, without typing the prompt itself. An empty line ends an indented function or loop at `...`. Keep this interpreter open through the container examples; use `exit()` before running another terminal command.

The chapter examples run in order without completing any repository TODO. Use the Week 1 `applied-programming-2026` environment and a separate practice folder, not the submission repository. If that environment is missing, use only the setup instructions in Assignment Steps 1-2, then return here; do not start the TODOs. Pages that show an expected result are not extra commands to copy.

```
start = (0.0, 0.0)
goal = (3.0, 4.0)

dx = goal[0] - start[0]
dy = goal[1] - start[1]
squared_length = dx * dx + dy * dy

def finish_distance(value):
    return value**0.5

distance = finish_distance(squared_length)
print(distance)
```

```
5.0
```

The names `dx` and `dy` retain the two coordinate differences. The expression inside `finish_distance` creates a numerical value, and `return` makes that value available to the caller. By contrast, `print` writes text to the terminal. A test can compare a returned number directly; it should not have to recover the number from display text.

Construct	Meaning in this program
name	a label bound to a Python object
assignment	evaluate the right side, then bind the result
expression	produce a value
function call	bind arguments, execute the body, and produce the returned value
<code>print(...)</code>	display text as a side effect; it is not a numerical return value

The 3-4-5 calculation keeps every intermediate value easy to check. The same values will next become a waypoint list, a tested function, and an array calculation.

A tuple records one point; a list records the ordered path

The start and goal above each store two coordinates. A tuple can record this ordered pair, while a list can collect an ordered path. A tuple's entries cannot be reassigned, whereas a list can be extended or edited. Python does not know that the first slot means x and the second means y; that interpretation belongs to the course contract.

```
start = (0.0, 0.0)
corner = (3.0, 0.0)
goal = (3.0, 4.0)
waypoints = [start, corner]
waypoints.append(goal)

x_goal, y_goal = goal
first_two = waypoints[:2]
print(waypoints[1], first_two, x_goal, y_goal)
```

```
(3.0, 0.0) [(0.0, 0.0), (3.0, 0.0)] 3.0 4.0
```

Indexing selects one item, so `waypoints[1]` is the second waypoint. Slicing selects a range and excludes the stop index, so `[:2]` contains indices 0 and 1. Unpacking binds the two tuple positions to two names in the same order. `append` changes the list by adding one waypoint at its end.

Python operation	Path interpretation
<code>point = (x, y)</code>	one ordered tool-center point
<code>waypoints = [...]</code>	one ordered waypoint sequence
<code>waypoints[i]</code>	waypoint i
<code>waypoints[a:b]</code>	path prefix or subpath from a through b-1
<code>x, y = point</code>	unpack two positions without changing their order
<code>waypoints.append(point)</code>	extend the sequence by one waypoint

Writing `point = x_goal, y_goal` packs a tuple; `x, y = point` unpacks it. A tuple is not automatically a point, nor a list automatically a path. Coordinate and waypoint order supply that meaning.

Iteration exposes the N waypoints to N - 1 segment relation

Continue with the three-entry `waypoints` list above. The two slices below have equal length and differ by one starting position. `zip` pairs their entries, giving the first-to-second and then second-to-third waypoint. Each `for p0, p1` unpacks one pair. Three waypoints therefore produce two segments.

```

segments = []
for p0, p1 in zip(waypoints[:-1], waypoints[1:]):
    displacement = (p1[0] - p0[0], p1[1] - p0[1])
    segments.append(displacement)

for index, displacement in enumerate(segments):
    print(index, displacement)

segments_comp = [
    (p1[0] - p0[0], p1[1] - p0[1])
    for p0, p1 in zip(waypoints[:-1], waypoints[1:])
]

```

```

0 (3.0, 0.0)
1 (0.0, 4.0)

```

The loop and comprehension express the same ordered calculation. `enumerate` supplies the index and value together. Alternatively, `range(len(segments))` supplies indices 0 and 1, which then require indexing the list. Keeping the value alongside its index avoids that extra lookup when both are needed.

```

import numpy as np

if len(waypoints) < 2:
    raise ValueError("a path needs at least two waypoints")

path = np.asarray(waypoints, dtype=np.float64)
print(path.shape)

```

```

(3, 2)

```

The `if` statement checks a condition; `raise` reports an exception instead of continuing with unusable data. Here the list is valid, so conversion proceeds. `np.asarray` collects its coordinates into one numerical array, and `dtype=np.float64` requests floating-point storage. The shape `(3, 2)` records three waypoints with two coordinates each. We will examine array storage in detail after making one calculation safe to call and test.

2.2 From one calculation to a reusable function

The review produced a distance and a list of segment displacements. Reusing that work requires more than moving the arithmetic below `def`. Another part of a program must know which arguments to supply, what a returned value means, and what happens when its data are invalid. These promises form a function contract.

```

def segment_length(start_xy, goal_xy) -> float:
    """Return Euclidean length in course coordinate units.

    Inputs are two finite real points ordered as (x, y).
    Raise PathInputError for invalid path data.
    """
    ...

```

This is the Assignment interface, not a runnable implementation: the ellipsis reserves the body. A signature gives the function name and parameters. Positional arguments match their order; keyword arguments name the parameter

directly. The annotation `-> float` describes the intended return type, while the docstring explains meaning and units. Neither executes a validity check.

Contract part	Published meaning
input	two real, finite points, normalized to shape <code>(2,)</code>
output / unit	one finite, nonnegative Python float in course coordinate units
valid boundary	identical endpoints return zero
invalid input	raise <code>PathInputError</code> ; do not clip or return a sentinel
side effects	do not print, write files, or change caller-owned inputs

A public function is one that code outside its module is expected to call. It is not an extra file and has nothing to do with making a GitHub repository public. The four public functions will remain in the one supplied module. Before implementing that full interface, we will develop a smaller, complete example using the tuple/list points already understood.

Type hints document a promise; they do not enforce it

```
def echo_coordinate(value: float) -> float:
    return value

print(echo_coordinate("not a number"))
```

```
not a number
```

Python executes this call even though the string conflicts with the annotation. Static checking tools can report the mismatch, but an annotation alone does not reject it during execution. A docstring is also descriptive text, not a guard. To keep invalid values out of arithmetic, we must write branches that inspect the actual arguments.

Validate familiar values before introducing array machinery

For a point, `(3.0, 4.0)` has two usable coordinates. `(3.0,)` is missing one; `(True, 4.0)` substitutes a truth value; `(float("nan"), 4.0)` contains a value that cannot represent a finite coordinate. These are different causes, so the checks should identify them before subtracting coordinates.

The next example accepts only a tuple/list of two Python integers or floats. This limited teaching contract is intentional; NumPy arrays and NumPy scalar types are added later. In Anaconda Prompt, enter a separate folder named `week02_practice` outside the submission repository. Create that folder only if it does not already exist. Open `learning_geometry.py` there, paste the complete listing below, and save it. This practice module is not a submission file.

```
notepad learning_geometry.py
```

`PathInputError` is a named kind of `ValueError`, used to report invalid path data. The short class declaration supplies that name; designing classes is next week's topic. The Assignment already provides its exception class. Here we define a separate practice copy so this file works without importing an unfinished package.

```
import math

class PathInputError(ValueError):
    """Report invalid path data."""

def _point_pair(value):
    if not isinstance(value, (tuple, list)):
        raise PathInputError("point must be a tuple or list")
    if len(value) != 2:
        raise PathInputError("point needs two coordinates")
    for coordinate in value:
        if isinstance(coordinate, bool):
            raise PathInputError("Boolean is not a coordinate")
        if type(coordinate) not in (int, float):
            raise PathInputError("coordinate must be real")
    try:
        x, y = float(value[0]), float(value[1])
    except (TypeError, ValueError, OverflowError) as exc:
        raise PathInputError("coordinate cannot become a float") from exc
    if not math.isfinite(x) or not math.isfinite(y):
        raise PathInputError("coordinates must be finite")
    return x, y
```

The leading underscore marks `_point_pair` as a helper used inside this module. It checks the container before calling `len`, then checks the count before indexing. `isinstance` recognizes a type and its subtypes; `type(coordinate)` identifies the exact type. The latter restricts this early example to Python int/float. Boolean receives its own error message: its relationship to int in Python does not make it a coordinate.

Connect the checks to the calculation and trace the result

The helper converts accepted values to floats so the calculation receives a consistent representation. A very large integer can fail that conversion. A `try` block attempts it, and `except` translates the listed conversion errors into `PathInputError`. The `from exc` clause retains the original cause for diagnosis. Then `math.isfinite` rejects infinity and NaN. Only a valid pair reaches `return x, y`.

Append the following function beneath the helper in the same file and save. The caller invokes the helper once for each endpoint. Both calls finish before the displacement is calculated. `math.hypot(dx, dy)` evaluates Euclidean distance, the same square-root relation used in the 3-4-5 example. A final check rejects a non-finite result even if each input was finite.

```
def distance_example(start_xy, goal_xy) -> float:
    """Return distance for two Python tuple/list points."""
    x0, y0 = _point_pair(start_xy)
    x1, y1 = _point_pair(goal_xy)
    result = math.hypot(x1 - x0, y1 - y0)
    if not math.isfinite(result):
        raise PathInputError("distance must be finite")
    return result
```

Open a fresh Python interpreter from this practice folder and run the following calls. They use the same saved function: the first supplies arguments by position, the second by name, and the third unpacks a pair of points with `*pair`. Without the star, that last call would supply one argument instead of two.

```
from learning_geometry import distance_example

print(distance_example((0.0, 0.0), (3.0, 4.0)))
print(distance_example(goal_xy=(3.0, 4.0), start_xy=(0.0, 0.0)))
pair = ((0.0, 0.0), (3.0, 4.0))
print(distance_example(*pair))
```

```
5.0
5.0
5.0
```

The following loop then passes one normal point and three invalid points through the same caller. Its exception handler prints a message so that all four trials can be observed; it does not change the numerical function or turn invalid input into a distance.

```
from learning_geometry import distance_example, PathInputError

for goal in [(3.0, 4.0), (True, 4.0), (3.0,), (float("nan"), 4.0)]:
    try:
        print(distance_example((0.0, 0.0), goal))
    except PathInputError as error:
        print(error)
```

```
5.0
Boolean is not a coordinate
point needs two coordinates
coordinates must be finite
```

The first call reaches the arithmetic and returns 5.0. The Boolean, missing coordinate, and NaN stop at different guards. None produces a fallback zero. Repeating these calls manually reveals the behavior; recording expectations as tests will make the same check repeatable after every edit.

Keep calculation inside a module and display outside it

The saved `learning_geometry.py` file is now a module: a Python file whose names can be imported. The call blocks printed results outside the function. This separation matters because a test, a later planner, and a display program need the returned number, not a line of terminal text.

`from learning_geometry import distance_example` makes the function available to the caller. The call blocks above then execute its body. Keep only imports and definitions in the practice module, leaving calls in the interpreter. After editing and saving the module, exit and restart Python before importing it again, so the next trial uses the updated definitions.

A pure numerical function bases its result on its inputs and does not change outside state. Printing, saving a file, and modifying an input array are side effects. Returning a new number or array is not itself such a modification. The supplied package preserves this division at a larger scale:

Component	Responsibility
<code>path_geometry.py</code>	validate arguments and return the four numerical results
student tests and engineering note	check behavior and explain observed results
provided demo and display scripts	call the numerical functions and write/show their results
generated JSON, SVG, wheel	numerical summary, browser preview, and installable package

The Assignment uses `from ap_week02_path.path_geometry import segment_length`: package, module, then function. The provided in-package code can use a relative import beginning with `..`. These imports locate code; they do not certify its correctness. We will run the display and packaging scripts only after implementing the four functions in the Assignment. For now, the practice module is sufficient for testing.

2.3 Tests turn an expected behavior into an executable check

A printed 5.0 is useful only because the 3-4-5 geometry tells us what to expect. A unit test records that reasoning as a call followed by an assertion. The assertion checks a stated relation and fails if it is false. These examples are deterministic tests: the selected input and expected behavior do not depend on random data, a clock, a display, or a simulator.

Test role	Chosen case	Expected behavior
normal	(0,0) to (3,4)	ordinary distance is 5
boundary	identical endpoints	zero length is allowed
invalid	Boolean coordinate	<code>PathInputError</code> , not a numerical result

Save the following as `test_learning_geometry.py` beside the practice module. This complete example tests the function already written above; it does not require any Assignment TODO. `pytest` discovers functions whose names begin with `test_`. Each function expresses one expectation.

```
import pytest
from learning_geometry import distance_example, PathInputError

def test_normal_distance():
    actual = distance_example((0.0, 0.0), (3.0, 4.0))
    assert actual == pytest.approx(5.0, rel=1e-12, abs=1e-12)

def test_zero_length():
    assert distance_example((2.0, 1.0), (2.0, 1.0)) == 0.0

def test_invalid_point():
    with pytest.raises(PathInputError):
        distance_example((0.0, 0.0), (True, 4.0))
```

`pytest.approx` allows the explicitly stated small floating-point difference; the tolerance is explained next. The exact zero comparison checks a boundary guaranteed by this example. The `with pytest.raises(...)` block passes only when the enclosed call raises the specified exception. Returning a number would fail that test.

```
python -m pytest -q test_learning_geometry.py
```

```
3 passed
```

Run this command in Anaconda Prompt from the practice folder, not at the Python prompt. `python -m pytest` starts pytest using the active interpreter, and `-q` requests a compact report. Three passes establish these three checks, not correctness for all possible inputs. The Assignment later requires independent cases rather than copying these test bodies.

Trace a returned value that disagrees with the test

To see what a genuine wrong-result failure means, use a separate deliberate mistake. The next function adds horizontal and vertical travel instead of calculating direct Euclidean distance. Save it as `example_wrong_distance.py` in the practice folder. It is a teaching counterexample, not the supplied starter and not a submission file.

```
import pytest

def wrong_distance(start, goal):
    return abs(goal[0] - start[0]) + abs(goal[1] - start[1])

def test_wrong_distance():
    actual = wrong_distance((0.0, 0.0), (3.0, 4.0))
    assert actual == pytest.approx(5.0, rel=1e-12, abs=1e-12)
```

```
python -m pytest -q example_wrong_distance.py -x
```

```
E      assert 7.0 == 5.0 ± 5.0e-12
E          Obtained: 7.0
E          Expected: 5.0 ± 5.0e-12
```

Read the failing test name, the assertion, the obtained and expected values, and the referenced source expression. The function returned $3 + 4 = 7$, while the test asks for direct distance 5. The result is valid Python output but answers the wrong geometric question. Correct the calculation rather than relaxing the expectation. The printed E lines explain this one mismatch.

An unfinished starter is a different kind of failure

The Assignment starts from deliberately unimplemented functions. Its recorded full baseline is **42 failed, 6 passed**; a failed call raises `NotImplementedError` instead of producing a distance. This is a preview of what you will run in Assignment Step 2, not a result required from the practice module.

```
python -m pytest -q tests/test_published_contract.py -k segment_length -x
```

```
E      NotImplementedError: Implement segment_length
```

`-k segment_length` selects matching test names. `deselected` counts excluded tests, not passed tests. `-x` stops at the first failure. An E prefix marks a failure-detail line, not an extra failing test. With an unfinished TODO, implement the missing behavior; with a returned but incorrect value, trace the expression that produced it.

A tolerance states which numerical differences are acceptable

A missing function and a wrong formula are not rounding errors. Once the formula is correct, small differences can remain because binary floating-point does not represent every decimal exactly. A tolerance states how large a difference this comparison will accept. In a fresh Python interpreter, the following arrays simply group two trial values so their outcomes can be compared together.

$$|actual - expected| \leq atol + rtol \times |expected|$$

For finite values, `np.isclose(actual, expected, ...)` uses the second value as the reference. The absolute tolerance `atol` has the same unit as the result; the relative tolerance `rtol` is a dimensionless fraction of the reference magnitude. This comparison is generally not symmetric.

```
import numpy as np

actual = np.array([5.000004, 5.000020])
print(np.isclose(actual, 5.0, rtol=1e-6, atol=1e-8))

near_zero = np.array([5e-10, 5e-8])
print(np.isclose(near_zero, 0.0, rtol=1e-6, atol=1e-9))
```

```
[ True False]
[ True False]
```

In the first comparison the allowance is 0.00000501: the error 0.000004 passes and 0.000020 fails. With reference zero, the relative term is zero, so only the absolute allowance 0.000000001 remains. An arbitrarily large `atol` could conceal a defect near zero; an unnecessarily tiny one could reject harmless rounding.

For the later course-coordinate examples we explicitly use `rtol=1e-12`, `atol=1e-12`. These settings compare numerical calculations; they are not a physical measurement accuracy or a universal default. When comparing arrays, also require equal shapes. A library may compare compatible but different shapes by expanding dimensions, a behavior examined in the next section.

```
expected = np.array([7.0, 10.0])
actual = np.array([7.0 + 1e-13, 10.0])
assert actual.shape == expected.shape
print(np.allclose(actual, expected, rtol=1e-12, atol=1e-12))
# True: every element passes the numerical comparison
```

`np.allclose` returns one Boolean for the complete array. In contrast, `pytest.approx` accepts a finite result if either its absolute or relative allowance is met; it does not add them. The earlier test therefore displayed $5.0 \pm 5.0e-12$. Use each tool's own documented rule rather than treating their parameters as interchangeable formulas.

2.4 NumPy makes point, waypoint, and batch axes explicit

The tests now describe one useful function, but a tool path contains many points and a study may contain many paths. NumPy stores their numerical coordinates in one array so a single expression can operate on many entries. The array must still preserve the meanings introduced by the tuple/list review. Its `ndim` is the number of axes, and `shape` gives the length of each axis. These lengths do not themselves say which axis means waypoint or coordinate; the program must define that interpretation.

one point → one path → a batch of paths

point	path	paths
(2,)	(N, 2)	(B, N, 2)
x, y	waypoint, coordinate	path, waypoint, coordinate

Figure 2.2. Shape is part of the data contract, not only a storage detail.

```
# Fresh Python session
import numpy as np
path = np.asarray(
    [(0.0, 0.0), (3.0, 0.0), (3.0, 4.0)],
    dtype=np.float64,
)

print(path.ndim)
print(path.shape)
print(path[1, 0])
```

```
2
(3, 2)
3.0
```

Representation	Shape	Axis meaning
point	(2,)	axis 0: x and y
path	(N,2)	axis 0: waypoint; axis 1: coordinate
batch	(B,N,2)	axis 0: path; axis 1: waypoint; axis 2: coordinate

For `path[1, 0]`, the first index chooses waypoint 1 and the second chooses its x coordinate, giving 3.0. A `dtype` specifies the array's element representation; `float64` stores numerical values as 64-bit floating-point numbers. This storage choice is distinct from shape. A two-row column with shape (2,1) is not a two-coordinate point with shape (2,).

Extend the same validation logic to arrays

The earlier helper assumed a tuple/list and inspected its original Python items. Array conversion introduces another issue: converting a mixed sequence can erase information about the input. The following example turns `True` into `1.0`. Checking only the converted number would accept an input the contract rejects.

```
import numpy as np

print(np.asarray([True, 4.0]))
raw = np.asarray([True, 4.0], dtype=object)
print(isinstance(raw[0], bool))
print(np.asarray([3.0, 4.0]).shape)
print(np.asarray([[3.0], [4.0]]).shape)
```

```
[1. 4.]
True
(2,)
(2, 1)
```

A temporary `dtype=object` array retains references to the original items, so their types can be examined before numerical conversion. The Assignment still rejects an incoming object-dtype ndarray; a temporary internal inspection array is a different object with a different purpose. Add the next two parts, in order, to `learning_geometry.py`. They form one helper, not two functions.

```
import numpy as np

def _point_array(value):
    if isinstance(value, np.ndarray) and value.dtype == object:
        raise PathInputError("input array must have a numerical dtype")
    try:
        raw = np.asarray(value, dtype=object)
    except (TypeError, ValueError, OverflowError) as exc:
        raise PathInputError("point cannot form an array") from exc
    if raw.shape != (2,):
        raise PathInputError("point must have shape (2,)")
```

Container conversion is attempted inside `try`. An unsupported or malformed conversion is reported through the same path-data exception. A successfully created array can still have the wrong shape, so the next guard requires exactly (2,). Only then are its items inspected:

```
number_types = (int, float, np.integer, np.floating)
for item in raw.flat:
    if isinstance(item, (bool, np.bool_)):
        raise PathInputError("Boolean is not a coordinate")
    if not isinstance(item, number_types):
        raise PathInputError("coordinate must be real")
try:
    point = raw.astype(np.float64)
except (TypeError, ValueError, OverflowError) as exc:
    raise PathInputError("coordinate cannot use float64") from exc
if not np.all(np.isfinite(point)):
    raise PathInputError("coordinates must be finite")
return point
```

`raw.flat` visits stored items. The explicit loop rejects Boolean before accepting Python or NumPy integer/floating scalar types; strings and complex values also fail. `astype(np.float64)` then creates the numerical representation. `np.isfinite` checks each element, and `np.all` requires every result to be true. The order remains representation, shape, type, conversion, finiteness; the original array is never written to.

Append this caller beneath the helper. It changes how input is accepted, not what distance means. The earlier tuple/list-only function remains available for comparison.

```
def distance_array_example(start_xy, goal_xy) -> float:
    start = _point_array(start_xy)
    goal = _point_array(goal_xy)
    result = math.hypot(goal[0] - start[0], goal[1] - start[1])
    if not math.isfinite(result):
        raise PathInputError("distance must be finite")
    return result
```

Save, restart Python in the practice folder, import NumPy, and run the trace below. A tuple and numerical ndarray now both work. A column-shaped array, Boolean coordinate, and NaN fail for separate reasons.

```
import numpy as np

from learning_geometry import distance_array_example, PathInputError

goals = [(3.0, 4.0), np.array([3.0, 4.0]),
         [[3.0], [4.0]], [True, 4.0], [3.0, float("nan")]]
for goal in goals:
    try:
        print(distance_array_example((0.0, 0.0), goal))
    except PathInputError as error:
        print(error)
```

```
5.0
5.0
point must have shape (2,)
Boolean is not a coordinate
coordinates must be finite
```

A slice may share data with the caller

Validation gives the values a usable representation, but ownership still matters. A NumPy slice can refer to the original storage rather than copy it. If a numerical function writes through such a view, it can change the caller's path even when it returns the right length.

```
path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
middle_view = path[1:2]
middle_view[0, 0] = 99.0
print(path)
```

```
[[ 0.  0.]
 [99.  0.]
 [ 3.  4.]]
```

The slice `1:2` selects one row while retaining a waypoint axis of length one. Writing `99.0` through that view changes the original middle waypoint. `copy()` instead makes independent storage. A calculation that only reads inputs need not copy everything; what matters is not writing into shared input data.

Expression	Shape	Meaning
<code>path[1]</code>	<code>(2,)</code>	one waypoint, sharing data
<code>path[1:2]</code>	<code>(1,2)</code>	one-row view
<code>path[1:2].copy()</code>	<code>(1,2)</code>	independent one-row array

Reset the example before checking the function. Preserve a snapshot, call the array-aware distance function, and compare the input with its snapshot afterwards. This tests ownership separately from numerical distance.

```
from learning_geometry import distance_array_example

candidate = np.array([[0.0, 0.0], [3.0, 4.0]])
snapshot = candidate.copy()
result = distance_array_example(candidate[0], candidate[1])
assert np.isclose(result, 5.0, rtol=1e-12, atol=1e-12)
assert np.array_equal(candidate, snapshot)
print(result)
```

```
5.0
```

Broadcasting evaluates all interpolation parameters in one expression

We can now accept and protect two endpoint arrays, but two positions alone do not show intermediate locations along their connecting segment. The supplied path preview needs sample points between waypoints. Linear interpolation supplies those points without finding a new route or deciding movement timing. A dimensionless parameter α moves from 0 at the start to 1 at the goal; the output has the same course coordinate units as the endpoints.

$$p(\alpha) = (1 - \alpha) p_{start} + \alpha p_{goal} \quad 0 \leq \alpha \leq 1$$

A column of M α values has shape $(M,1)$, while one two-coordinate displacement has shape $(1,2)$. Broadcasting lets the expression combine every α with both displacement coordinates. It compares dimensions from the right: matching lengths or a length of one are compatible. Here the result is $(M,2)$, not an arbitrary repetition of the input path.

```
# Continue with numpy imported; this is the numerical kernel.
start = np.asarray((0.0, 0.0), dtype=np.float64) # (2,)
goal = np.asarray((3.0, 4.0), dtype=np.float64)  # (2,)
alpha = np.linspace(0.0, 1.0, 5)[: , None]      # (5, 1)

points = start[None, :] + alpha * (goal - start)[None, :]
points[0] = start
points[-1] = goal
print(points.shape)
print(points)
```

```
(5, 2)
[[0.  0. ]
 [0.75 1. ]
 [1.5  2. ]
 [2.25 3. ]
 [3.  4. ]]
```

`None` inserts an axis of length one. Broadcasting compares dimensions from the right: they must be equal or one must be 1. Here the output is `(5, 2)`. The two explicit row assignments preserve the normalized endpoints exactly even for decimal coordinates where arithmetic can round the last row differently. They change only the new output, never `start` or `goal`.

The sample count includes both endpoints

The displayed five rows consist of the start, three interior points, and the goal. Thus `num_samples` counts all returned rows. It must be an integer at least two; even identical endpoints do not waive that published minimum. Boolean is rejected because it is a truth value, not an explicit sample count.

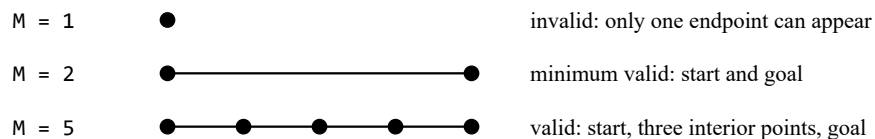


Figure 2.3. The sample count includes both endpoints of a segment with distinct start and goal positions.

Append this function to the practice module. It combines the point helper, the sample-count guards, and the already traced interpolation expression. The first and last rows are explicitly assigned from the normalized inputs. This makes their equality exact even when decimal arithmetic would otherwise round the final row.

```
def interpolate_example(start_xy, goal_xy, num_samples):
    start = _point_array(start_xy)
    goal = _point_array(goal_xy)
    if isinstance(num_samples, (bool, np.bool_)):
        raise PathInputError("sample count cannot be Boolean")
    if not isinstance(num_samples, (int, np.integer)):
        raise PathInputError("sample count must be an integer")
    if int(num_samples) < 2:
        raise PathInputError("sample count must be at least two")
    alpha = np.linspace(0.0, 1.0, int(num_samples))[1:, None]
    points = start[None, :] + alpha * (goal - start)[None, :]
    points[0] = start
    points[-1] = goal
    if not np.all(np.isfinite(points)):
        raise PathInputError("interpolated points must be finite")
    return points
```

Save and restart Python, then import NumPy and run the following decimal-coordinate example. Exact endpoint checks serve a different purpose from a tolerance-based comparison of computed interior points. These assignments change only the new output, never the input endpoints.

```
import numpy as np

from learning_geometry import interpolate_example

start = np.array([1.0, 1.0])
goal = np.array([0.1, 0.1])
points = interpolate_example(start, goal, 3)
assert points.shape == (3, 2)
assert np.array_equal(points[0], start)
assert np.array_equal(points[-1], goal)
print(points)
```

```
[[1.  1. ]
 [0.55 0.55]
 [0.1  0.1 ]]
```

Calls with 1, a non-integer, or Boolean must raise `PathInputError`; 2 returns just the endpoints. More samples describe the same straight segment more densely. They do not prove collision avoidance, change the geometric route, or set the robot's speed.

2.5 From one segment to a path and a batch

Interpolation samples one segment, while a polyline follows all its ordered waypoints. Measuring that polyline therefore brings us back to the adjacent pairs from the Python review. Its length is the sum of their Euclidean lengths, not generally the direct distance between the first and last point.

$$P = [p_0, p_1, \dots, p_{N-1}], N \geq 2$$

$$L(P) = \sum_{i=0}^{N-2} \|p_{i+1} - p_i\|_2$$

Here P is an ordered path with N waypoints, p_i is waypoint i , and the norm gives one segment length. Each term and the total have course coordinate units. Append this scalar reference to the practice module. Its precondition is a validated, normalized float64 array of shape $(N, 2)$, N at least two. Integer inputs remain valid for the public

functions, but must be converted before these kernels: fixed-width integer squaring can overflow without producing infinity. Validation, normalization, and calculation have distinct jobs.

```
def path_loop_example(path):
    """Kernel for a validated, normalized float64 array (N, 2)."""
    total = 0.0
    for p0, p1 in zip(path[:-1], path[1:]):
        total += distance_array_example(p0, p1)
    if not math.isfinite(total):
        raise PathInputError("path length must be finite")
    return total
```

After saving, restart Python in the practice folder and run:

```
import numpy as np
from learning_geometry import distance_array_example, path_loop_example

path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
print(distance_array_example(path[0], path[-1]))
print(path_loop_example(path))
```

```
5.0
7.0
```

The direct segment has length 5.0. The polyline goes three units to the corner, then four to the goal, giving 7.0. The earlier wrong-distance function returned 7 for the direct-distance question; this sum returns 7 for the different, explicitly requested path-length question. Correctness depends on the meaning of the requested result, not whether a familiar number appears.

Repeated waypoints are valid and contribute zero. The Assignment's public `path_length` must also reject malformed, non-finite, empty, or one-waypoint paths before this arithmetic. A complete path validator applies the same representation/type/finiteness checks already developed, but requires shape (N,2), not (2,). Its calculated total must also be finite.

Vectorization performs the same calculation across rows

The scalar loop exposes each segment separately. NumPy can calculate those same displacements together: subtracting the two shifted slices pairs each waypoint with its successor. Vectorization changes the execution form, not the geometry. Append the following numerical kernel to the practice module; its input has the same already-validated precondition as the scalar reference.

```
def path_array_example(path):
    """Kernel for a validated, normalized float64 array (N, 2)."""
    displacement = path[1:, :] - path[:-1, :]
    squared = displacement * displacement
    segment_lengths = np.sqrt(np.sum(squared, axis=1))
    result = float(np.sum(segment_lengths))
    if not np.isfinite(result):
        raise PathInputError("path length must be finite")
    return result
```

Expression	Shape	Meaning
<code>path[1:] - path[:-1]</code>	<code>(N-1,2)</code>	one displacement per segment
<code>sum(..., axis=1)</code>	<code>(N-1,)</code>	squared length of each segment
<code>sqrt(...)</code>	<code>(N-1,)</code>	Euclidean segment lengths
final sum	scalar	total polyline length

In `sum(squared, axis=1)`, the coordinate axis disappears: x-squared and y-squared become one squared length per segment. The square root gives segment lengths, and the final sum combines segments. Reducing axis 0 at the first step would instead mix different segments by coordinate and answer a different question.

The mathematical relation is the same, but finite-precision intermediates can differ. Squaring extremely large displacements can overflow even when `math.hypot` can return a finite distance. This kernel rejects a non-finite result; the following equivalence checks use the supplied moderate coordinate values, not a promise of identical behavior for every representable input.

Save and restart Python before importing the updated module. Compare the two implementations on the same independent 3 + 4 reference:

```
import numpy as np
from learning_geometry import path_array_example, path_loop_example

path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
actual = path_array_example(path)
expected = path_loop_example(path)
assert np.isclose(actual, expected, rtol=1e-12, atol=1e-12)
assert np.isclose(actual, 7.0, rtol=1e-12, atol=1e-12)
print(actual)
```

7.0

A batch adds a path axis while retaining the same meaning

One call now calculates one path. To compare several equally sized paths, collect their waypoint arrays along a new first axis. B paths with N two-coordinate waypoints have shape `(B,N,2)`. Append this kernel to the practice module. It assumes validated, normalized float64 input, B at least one and N at least two; it removes the coordinate and segment axes while retaining one output per path.

```
def batch_example(paths):
    """Kernel for a validated, normalized float64 array (B, N, 2)."""
    displacement = paths[:, 1:, :] - paths[:, :-1, :]
    segment_lengths = np.sqrt(np.sum(displacement**2, axis=2))
    lengths = np.sum(segment_lengths, axis=1)
    if not np.all(np.isfinite(lengths)):
        raise PathInputError("batch lengths must be finite")
    return lengths
```

The expression `paths[:, 1:, :] - paths[:, :-1, :]` keeps every path and both coordinates while pairing consecutive waypoints. Reduction along axis 2 combines x and y; reduction along axis 1 then combines segments. The batch axis, axis 0, survives. Save, restart Python, and supply these two complete paths:

```
import numpy as np
from learning_geometry import batch_example

paths = np.array([
    [[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]],
    [[1.0, 1.0], [7.0, 1.0], [7.0, 5.0]],
], dtype=np.float64)
print(paths.shape)
lengths = batch_example(paths)
print(lengths.shape)
print(lengths)
```

```
(2, 3, 2)
(2,)
[ 7. 10.]
```

The first path travels $3 + 4 = 7$ units; the second travels $6 + 4 = 10$. The two output entries correspond to the two input paths in the same order. The public `path_lengths` also validates shape $(B,N,2)$, B at least one, N at least two, allowed real types, and finite input before the kernel. An empty batch or one-waypoint path is invalid. This teaching kernel assumes those checks; it is not by itself the full Assignment solution.

Test relationships that should survive a change of input

The two expected lengths anchor the calculation, but a wrong implementation could accidentally match fixed examples. Additional checks change the input while preserving a relation we can reason about. Continue in the same interpreter with the two-path array just defined. The scalar loop supplies one comparison; translating and reordering paths supply different ones.

```
from learning_geometry import batch_example, path_loop_example

actual = batch_example(paths)
scalar = np.array([path_loop_example(row) for row in paths])
assert actual.shape == (2,)
assert np.allclose(actual, [7.0, 10.0], rtol=1e-12, atol=1e-12)
assert np.allclose(actual, scalar, rtol=1e-12, atol=1e-12)

shift = np.array([10.0, -2.0])
translated = batch_example(paths + shift)
assert np.allclose(translated, actual, rtol=1e-12, atol=1e-12)

snapshot = paths.copy()
_ = batch_example(paths)
assert np.array_equal(paths, snapshot)
permuted = batch_example(paths[::-1])
assert np.allclose(permuted, actual[::-1], rtol=1e-12, atol=1e-12)
print("All four consistency checks passed")
```

```
All four consistency checks passed
```

Adding $(10,-2)$ to every point moves the paths without changing displacements, so lengths stay equal within the chosen tolerance. Reversing batch rows reverses the output order. Comparing the input with its saved snapshot checks that no coordinate was changed; exact equality is appropriate here.

The list comprehension belongs to the test reference, not to the vectorized batch kernel. Agreement between two implementations alone is insufficient if both contain the same defect, so retain hand-calculated values and transformation checks as complementary evidence. These are deterministic tests of selected properties, not exhaustive input coverage or a generative property-based testing method.

2.6 Exercises and transfer to the package

The practice module now connects the whole calculation: validated points, one distance, interpolation, a scalar path sum, an equivalent vectorized sum, and batch checks. The final task is to bring those ideas into the supplied package without changing its interface. The exercises below let you explain the reasoning before relying on its test report; they do not require completed repository TODOs.

Exercise 1

Trace the Python review's 3-4-5 calculation. Explain what the caller receives if `return value**0.5` is replaced by `print(value**0.5)`.

Exercise 2

Trace a valid point, a Boolean coordinate, and a one-column array through the relevant helper. State which check accepts or rejects each, and why converting immediately to float64 can conceal the Boolean input.

Exercise 3

Predict the rows of `interpolate_example((1,2), (5,6), 3)`. Identify which two rows establish the endpoint contract and explain why three means three total points, not three interior points.

Exercise 4

Construct two three-waypoint paths with the same endpoints but different lengths. Calculate the results, then trace the shapes in a (B,N,2) batch calculation. Explain why reducing the batch axis would lose the one-result-per-path meaning.

Exercise 5

Choose a case not based on the fixed 3-4-5 example. State its independently expected result or transformation relation, its numerical tolerance where appropriate, and the defect your test would reveal.

In the integrated Assignment, create the Week 2 private repository and record its untouched baseline before editing. Then transfer the calculation and validation ideas into the four designated TODOs. The practice module, teaching tests, and deliberate wrong example stay outside the submission. Full public contracts are broader than the early tuple/list example; the path and batch kernels still need the specified input validation.

Use the following numbered steps as the single submission route. If you already performed setup to obtain the environment, resume from the first incomplete step without recreating the repository or overwriting your note. There is no need to revisit an earlier teaching example merely to make it run: all chapter examples were independent of the unfinished package.

2.7 Assignment 2: Build and Verify a Path-Geometry Package

Complete the four numerical functions in `ap_week02_path.path_geometry`. They measure and interpolate a supplied two-dimensional tool path. Preserve coordinate order, reject invalid inputs with `PathInputError`, and leave caller-owned arrays unchanged. Another person must be able to install and test the revision you submit.

Figure 2.1 supplies the application: an ordered robot-tool path. You do not generate a route, check collisions, assign motion timing, implement a controller, or write a visualizer. The provided program calls your functions to produce a numerical summary and an animated SVG path preview. A public function means a function called by outside code, not a public repository or an extra file.

File you edit	Work inside that file
<code>src/ap_week02_path/path_geometry.py</code>	Complete TODO 1-4; add internal helpers as needed.
<code>tests/test_student_evidence.py</code>	Write at least six independent test functions.
<code>artifacts/engineering_note.md</code>	Write responses under the existing markers in sections 0-5.

Manually edit only these three files. Keep the public names, signatures, exception class, units, shapes, package metadata, published tests, and provided scripts unchanged. Generate JSON, SVG, and wheel files with the supplied commands; do not hand-edit them. Read `README.md` and `PROTECTED_FILES.md` with this Assignment before starting.

Step 1 - Create this week's private repository and invite the instructor

Use your existing verified GitHub account from Week 1. If necessary, create and verify an account at <https://github.com/signup> first. Open the template below, choose Use this template → Create a new repository, select your own account as owner, and choose Private. Name it `applied-programming-w02-<student-id>`, replacing the placeholder with your actual student ID. This is a separate Week 2 repository; do not rename or reuse the Week 1 repository.

```
https://github.com/SSUMechE/applied-programming-2026-week02-starter
```

In your new repository, open Settings → Collaborators → Add people. Search for and select the exact account SSUMechE, then send the collaborator invitation. A private URL alone does not give the instructor access. Pending means sent but not yet accepted; active access begins after acceptance. Continue working while it is pending and report the real state. A correct invitation sent on time and awaiting only instructor acceptance is not a student omission.

Step 2 - Clone, install, and record the untouched baseline

Open Anaconda Prompt, not the Python >>> prompt or PowerShell. Move to the parent folder where you keep course projects. Replace <your-account> and <student-id> in both lines below. Clone your private repository, not the public template. If that clone already exists, enter it with `cd`; do not create another repository.

```
git clone https://github.com/<your-account>/applied-programming-w02-<student-id>.git
cd applied-programming-w02-<student-id>
git remote -v
```

Both `origin` lines must identify your private Week 2 URL. Reuse the Week 1 environment. Only if it does not exist, run `conda env create -f environment.yml` here. If that command stops on a defaults-channel Terms of Service error, the following alternative selects only conda-forge with the same course versions. It does not accept terms or change your channel settings. Skip both creation commands when the environment already exists.

```
conda create -n applied-programming-2026 --override-channels -c conda-forge ^
python=3.12.13 pip=26.2.1
```

The ^ joins these two lines into one Anaconda Prompt command. Leave nothing after it. Confirm the installation prompt. Conda installs Python and pip only. The commands below activate the environment and install NumPy and the other required packages with pip. Run them from this repository root. Do not install NumPy separately with Conda.

```
conda activate applied-programming-2026
python -m pip install -r requirements.txt
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
```

The verifier must end with `[PASS] Week 2 path-geometry environment is ready`. It checks the installed runtime and package setup, not the unfinished functions' numerical correctness. Resolve an installation failure before editing TODOs; do not change package pins or protected scripts to bypass it.

```
python -m ap_week02_path.review_data_flow
python scripts/run_baseline.py
notepad artifacts\engineering_note.md
```

The review command traces Python containers and array conversion without supplying the four solutions. The baseline command runs the untouched published tests. Record `42 failed, 6 passed` and the full first `FAILED` test node ID in section 0 of the note. Keep the prompt and response marker, replace the placeholder, save, and close the editor. The baseline failures are expected: TODO calls deliberately raise `NotImplementedError`.

If the baseline count differs, first check the active environment, working folder, package installation, and whether source or tests were already edited. Do not invent a baseline record or overwrite completed work merely to recover the initial count.

Step 3 - Complete the four TODOs in dependency order

Open `src/ap_week02_path/path_geometry.py` in your editor. All four functions are in this one file. Validate inputs before arithmetic; return finite results rather than silently clipping values or returning a sentinel. Save each change before testing.

Order	Implementation and observable checkpoint
TODO 1: <code>segment_length</code>	Two real, finite $(2,)$ points $\rightarrow$ nonnegative Python float. $(0,0)$ to $(3,4)$ returns 5.0; identical endpoints return 0.0.
TODO 2: <code>interpolate_segment</code>	Python/NumPy integer $M \geq 2$, not Boolean $\rightarrow$ <code>float64 (M,2)</code> . $M=5$ returns $(5,2)$ with exact normalized start and goal rows.
TODO 3: <code>path_length</code>	Finite $(N,2)$, $N \geq 2 \rightarrow$ sum of consecutive lengths. $(0,0), (3,0), (3,4)$ has length 7.0, not endpoint distance 5.0.
TODO 4: <code>path_lengths</code>	Finite $(B,N,2)$, $B \geq 1, N \geq 2 \rightarrow$ <code>float64 (B,)</code> . Use NumPy, with no Python <code>for/while/comprehension</code> in this function.

After TODO 1, run the first selection below; after TODO 2, run the second. Then complete TODO 3 and TODO 4 with their respective selections. A selected test may still need a shared helper, so read the first failure rather than bypassing it.

```
python -m pytest -q tests/test_published_contract.py -k segment_length -x
python -m pytest -q tests/test_published_contract.py -k interpolation -x
python -m pytest -q tests/test_published_contract.py -k "path_length and not path_lengths" -x
python -m pytest -q tests/test_published_contract.py -k path_lengths -x
python -m pytest -q tests/test_published_contract.py
```

`-k` selects matching names, `-x` stops after the first failure, and `deselected` does not mean passed. The last command has no filter and must report **48 passed**. This completes the core implementation checkpoint, not the entire submission.

Reject wrong representation, shape, Boolean/string/complex coordinates, object-dtype arrays, non-finite inputs, and invalid sample counts with `PathInputError`. An empty batch, empty path axis, or one-waypoint path is invalid. Repeated waypoints are valid. The functions must not print, plot, write files, call the visualizer, or mutate their inputs.

Step 4 - Write and run six independent tests

Edit `tests/test_student_evidence.py`. Write at least six top-level synchronous `test_...` functions covering all categories below. Each calls a public function and checks a returned result or expected exception. Choose your own cases and expected relations; copying a published test body is not independent work.

Required prefix	What the test establishes
<code>test_normal_</code>	An independently chosen ordinary segment, interpolation, or path result.
<code>test_boundary_</code>	A zero-length segment or minimum valid sample/path size.
<code>test_invalid_</code>	Wrong shape, non-finite value, invalid count, or unsupported type raises <code>PathInputError</code> .
<code>test_endpoint_</code>	First and last interpolated rows preserve the normalized start and goal.
<code>test_translation_</code>	Adding one common offset to every point preserves length.
batch / ownership	Use <code>test_batch_</code> , <code>test_ownership_</code> , <code>test_no_mutation_</code> , or <code>test_scalar_batch_</code> for scalar-batch agreement or input ownership.

Use `np.isclose` or `np.allclose` with explicit `rtol` and `atol` for numerical evidence in normal, translation, and batch/ownership categories. Exact equality remains appropriate for shape, Boolean conditions, exact endpoint preservation, and unchanged snapshots. An ownership test can check both an explicitly tolerance-compared numerical result and an unchanged input snapshot.

Independent inputs may be built with variables, arrays, arithmetic, or helper functions; a fresh literal tuple or list inside every test is not required. What matters is a justified case and a real assertion, not the syntax used to construct it. Do not skip or mark the required tests as expected failures.

```
python -m pytest -q tests/test_student_evidence.py
python -m pytest -q
```

With six additional passing tests, the complete published-plus-student suite reports at least **54 passed**. A pass only establishes the checks actually executed. Explain why your cases and expected results are useful in the engineering note; the instructor reviews their meaning and independence, not just the count.

Step 5 - Generate results, complete the note, and check the wheel

Once the tests pass, the protected programs call your functions to create the numerical summary and path preview. Run them from the repository root, then open the existing note. Do not replace or recreate a note you have already started.

```
python -m ap_week02_path.demo
python scripts/generate_artifacts.py
start "" artifacts\path_geometry_preview.svg
notepad artifacts\engineering_note.md
```

Note section	Write under its existing response marker
0. Baseline	The recorded 42 failed/6 passed summary and first failing test node before TODO edits.
1. Public contract	How representation, shape, finite values, and function-specific rules are checked.
2. Numerical evidence	One independent normal and boundary case: inputs, expected meaning, output, tolerance, unit.
3. Scalar-batch	The comparison you ran, input/output shapes, and explicit tolerance.
4. Invariance / ownership	Translation and no-mutation checks, and what defect each would reveal.
5. Visualization	What the SVG displays and why it does not establish planning, collision checks, or robot dynamics.

Keep every question and `<!-- STUDENT RESPONSE -->` marker; replace the placeholder below each marker with your own response. Save with Ctrl+S, close the editor, and read the saved file back. Creating the file alone is not completion. There is no word-count minimum; answer the requested content accurately.

```
type artifacts\engineering_note.md
python -m build --wheel --no-isolation
python scripts/check_submission.py
```

Confirm the saved text includes your responses in all six sections. The checker examines tests and nonempty note responses, recomputes `artifacts/path_geometry_summary.json` and `artifacts/path_geometry_preview.svg`, and clean-installs the single wheel in `dist/`. Its PASS is an automated checklist, not a judgment that every explanation is correct. The instructor reviews explanation quality and independent-case suitability.

Step 6 - Commit and push the verified files

A saved file exists only on your computer until you record and push it. First rerun the checks and inspect the changes. `git diff` shows tracked edits; `git status --short` also identifies new files. Stage only the declared source, tests, note, generated outputs, and wheel.

```
python -m pytest -q
python scripts/check_submission.py
git status --short
git diff
git add src/ap_week02_path/path_geometry.py tests/test_student_evidence.py artifacts dist
git diff --staged
git commit -m "Complete Week 2 path-geometry package"
git push origin main
git status --short
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

Inspect `git diff --staged` before committing: it is the content about to enter the revision. `commit` records a local snapshot; `push` sends it to GitHub. Resolve any push error before claiming that the submitted revision is available remotely.

Check	Required observation
<code>git status --short</code> after push	No output: the local working tree is clean.
<code>git rev-parse HEAD</code>	Your local full 40-character commit identifier.
<code>git ls-remote origin refs/heads/main</code>	Its first field equals that same full identifier.
GitHub repository page	Your private Week 2 repository contains the pushed files.

Open the repository online and check the note and generated files at the pushed revision. Keep SSUMechE access active, or truthfully record that the correct on-time invitation is pending. Never commit credentials, tokens, recovery codes, environment folders, caches, or unrelated material.

If you change code or tests after verification, regenerate affected outputs, rebuild the wheel, run the checks again, and commit and push the new state. A branch name or the phrase 'latest version' is not a substitute for the full identifier used in the next step.

Step 7 - Submit the exact revision through LMS

For Week 2, use your private repository named `applied-programming-w02-<student-id>`. Follow the same GitHub and LMS submission procedure and common submission rules as in Week 1.

Completion checklist

- Environment verification passes.
- The untouched starter baseline and first failure are recorded.
- All four TODOs are complete.
- All 48 published tests and at least six student tests pass.
- The JSON summary and SVG preview have been regenerated.
- Sections 0-5 of `engineering_note.md` are answered.
- One wheel passes the clean-installation check.
- The private repository name and `origin` are correct.
- The revision is pushed, and the local and remote full commit IDs match.
- SSUMechE has access, or a correct invitation sent by the deadline is awaiting acceptance.
- All three required LMS fields are submitted.

The finished work is a reusable, tested numerical package with explained results and a reproducible source revision. Week 3 builds named domain objects on these tested functions, rather than repeating the calculations inside every class.